**FACULTY OF ENGINEERING AND BASIC SCIENCES**
**ACADEMIC PROGRAM: DATA ENGINEERING AND ARTIFICIAL INTELLIGENCE**

**COURSE: ETL (G01)**
**LAB-4. ETL Pipeline with Data Quality Validation (Great Expectations)**

## 1. Context

You and your team are working as data engineers at a retail company operating across Latin America. The business team needs a reliable analytics system to answer key questions about sales performance, customer behavior, and product demand across countries.

However, the available transactional data — captured directly from point-of-sale systems — contains multiple quality issues that make it unfit for analytics as-is:

- Missing and malformed values in key fields
- Duplicate transaction records inflating revenue figures
- Inconsistent country name formats across branches
- Negative quantities and prices from data entry errors
- Derived totals that do not match their source fields
- Transaction dates in mixed formats and out-of-range values

Ensuring data quality is therefore essential for trustworthy analytics and competitive advantage.

## 2. General Objective

Design and implement a complete ETL pipeline that incorporates data validation using Great Expectations, applies appropriate cleaning and transformation strategies, and produces reliable business insights from a dimensional model.

## 3. Learning Outcomes

By the end of this lab, students will be able to:

- design a modular ETL pipeline following professional project conventions
- implement validation rules using Great Expectations (Expectation Suites, Data Docs)
- identify, classify, and quantify data quality issues across six dimensions
- apply and justify data cleaning strategies per issue type
- distinguish between Cleaning and Transformation as separate pipeline stages
- validate data before and after transformation using the same GE framework
- design and implement a dimensional model (star schema) for a retail scenario
- connect data quality decisions to measurable business outcomes

## 4. Dataset

The dataset for this lab is provided as a CSV file: retail_etl_dataset.csv.
4.1 Column Description

| Column | Description | Expected Type | Expected Format / Range |
|---|---|---|---|

| | | | |
|---|---|---|---|
| invoice_id | Unique identifier for each transaction | Integer | Positive integer, unique |
| customer_id | Identifier of the customer who made purchase | Integer | Positive integer, not null |
| product | Name of the purchased product | String | From fixed catalog (8 items) |
| quantity | Number of units purchased | Integer | ≥ 1 |
| price | Unit price of the product at time of sale | Float | > 0.0 (USD) |
| total_revenue | Total amount = quantity × price | Float | > 0.0 (derived field) |
| country | Country where transaction occurred | String | Title case, from fixed set |
| invoice_date | Date the transaction took place | Date | YYYY-MM-DD, within 2023 |

4.2 Known Data Quality Issues

The following problems are present in the raw dataset. Students are expected to discover, quantify, and address all of them:

| Dimension | Column(s) | Problem | Approx. Count |
|---|---|---|---|
| **Completeness** | customer_id, invoice_date | NULL customer_id (as float NaN); null-like strings in invoice_date ('N/A', 'NULL', '') | 202 + 15 |
| **Uniqueness** | invoice_id | Duplicate invoice_id rows — same ID appears multiple times, inflating revenue | 1,159 (~23%) |
| **Validity** | quantity, price | Negative values in quantity (e.g. -3) and price (e.g. -83.02) — physically impossible | 149 + 101 |
| **Consistency** | country | Same country in multiple formats: 'Colombia', 'colombia', 'CO'; 'Ecuador', 'ecuador' | ~1,500 |
| **Accuracy** | total_revenue | total_revenue ≠ quantity × price in some rows (offset errors, wrong multiplier, sign flip) | 148 |

| | | | |
|---|---|---|---|
| **Timeliness** | invoice_date | Future dates (2025–2026); mixed formats: YYYY/MM/DD and DD-MM-YYYY alongside YYYY-MM-DD | 30 + 98 + 78 |

## 5. Business Objectives

Your ETL pipeline must support the following business goals. Each is referenced in the tasks below.

BO-1 — Ensure Financial Integrity
Guarantee that revenue reports accurately reflect real sales by eliminating duplicates, negative values, and inaccurate totals. This allows management to trust financial KPIs.

BO-2 — Support Strategic Planning with Reliable Trends
Standardize transaction dates and ensure completeness to enable time-based analyses (monthly growth, seasonal peaks). This helps managers optimize inventory and promotions.

BO-3 — Strengthen Product and Regional Insights
Ensure product names and country data are consistent and standardized. This allows teams to identify top-selling products and regional performance with confidence.

BO-4 — Enable Transparent and Defensible Reporting
Deliver clean, validated data with traceable quality decisions. This fosters trust among stakeholders by showing that reported figures are based on high-quality, auditable information.

## 6. Solution Architecture

Raw Data
↓
Extrac + Data Profiling
↓
Input Validation (GE)
↓
DQ Analysis
↓
Cleaning
↓
Transformation
↓
Post-Transformation Validation (GE)
↓
Dimensional Modeling (Star Schema)
↓
Load (Data Warehouse - local)
↓
Business Analysis

## 7. Tasks

### a. Extract & Profiling

~~Load the dataset into a Pandas DataFrame~~ and ~~register it as a Great Expectations data source~~ (in-memory Pandas datasource). Then perform systematic profiling to understand the raw data before writing any expectations.

Your profiling script must compute and report:

- ~~Shape (rows, columns), dtypes, memory usage~~
- ~~Missing value count~~ and percentage per column
- ~~Cardinality of all categorical columns (product, country, invoice_date formats)~~
- ~~Descriptive statistics for numeric columns (quantity, price, total_revenue): min, max, mean, median, std~~
- ~~Number of duplicate invoice_id values~~
- ~~Count of rows where total_revenue ≠ quantity × price (within tolerance ±0.01)~~
- ~~Distribution of invoice_date formats (YYYY-MM-DD vs YYYY/MM/DD vs DD-MM-YYYY vs other)~~
- ~~Count of future dates (> 2023-12-31) and null-like date strings~~

> *Deliverable: profiling summary table (one row per column) in your PDF report.*

### b. ~~Input Data Validation (Great Expectations)~~

~~**Business objectives:** BO-1, BO-2, BO-3, BO-4~~
~~Build a formal Expectation Suite for the raw input data. The purpose of this suite is to measure and document quality problems — most expectations are expected to FAIL. Failure rates are your quality baseline.~~
~~Your suite must include at least one expectation per quality dimension, covering all columns with known issues:~~

| Dimension | Column(s) | Minimum Expectation | Expected Result |
|---|---|---|---|
| Completeness | customer_id, invoice_date | expect_column_values_to_not_be_null | FAIL |
| Uniqueness | invoice_id | expect_column_values_to_be_unique | FAIL |
| Validity | quantity | expect_column_values_to_be_between(min=1) | FAIL |
| Validity | price | expect_column_values_to_be_between(min=0.01) | FAIL |
| Validity | product | expect_column_values_to_be_in_set(…) | FAIL |
| Accuracy | total_revenue | Custom: total_revenue == qty × price | FAIL |
| Consistency | country | expect_column_values_to_be_in_set(…) | FAIL |

| | | | |
|---|---|---|---|
| Timeliness | invoice_date | expect_column_values_to_match_regex(YYYY-MM-DD) | FAIL |
| Timeliness | invoice_date | expect_column_values_to_be_between('2023-01-01','2023-12-31') | FAIL |

Run validation on the raw data and generate a Data Docs report. Document the failure rate for each expectation.

> **Deliverables:** ~~*Expectation Suite JSON file; Data Docs HTML report; failure rate summary table in PDF.*~~

### c. Data Quality Analysis and Policy Proposal

**Business objectives:** BO-1, BO-2, BO-3, BO-4
Based on the profiling and validation results, produce two artifacts:

#### *c.1 Data Quality Issues Table*

Fill in a row for every issue found:

| Column | Issue | Example | Dimension | Business Impact |
|---|---|---|---|---|
| invoice_id | Duplicate IDs | 22951 appears 3× | Uniqueness | Revenue double/triple-counted in BO-1 KPIs |
| customer_id | NULL values | NaN in 202 rows | Completeness | Cannot link sales to customers for BO-3 |
| quantity | Negative values | -3 in row 5421 | Validity | Negative units corrupt total_revenue (BO-1) |
| price | | | | |
| total_revenue | | | | |
| country | | | | |
| invoice_date | | | | |
| invoice_date | | | | |

#### *c.2 Data Quality Policy Proposal*

Formalize at least 8 policies. Each must include a plain-English statement, a GE expectation, and a severity level. The first 6 are given as a starting template — you must add at least 2 original policies based on your own findings:

| # | Policy Statement | GE Expectation | Severity | Addresses (BO) |
|---|---|---|---|---|
| P-01 | invoice_id must be | expect_column_values_to_be_unique('invoice_id') | **Critical** | BO-1, BO-4 |

| | | | | |
|---|---|---|---|---|
| | unique across the entire dataset. | | | |
| P-02 | quantity must be a positive integer (≥ 1). | expect_column_values_to_be_between('quantity', min_value=1) | **Critical** | BO-1 |
| P-03 | price must be greater than zero. | | | |
| P-04 | total_revenue must equal quantity × price (±0.01 tolerance). | | | |
| P-05 | country must be one of: {Colombia, Ecuador, Peru, Chile} (title case, no abbreviations). | | | |
| P-06 | invoice_date must follow YYYY-MM-DD and fall within 2023-01-01 to 2023-12-31. | | | |
| P-07 | [Student-defined policy 1] | | | |
| P-08 | [Student-defined policy 2] | | | |

*Deliverables: Data Quality Issues table + Policy Proposal table in PDF report.*

### d. Cleaning

**Definition:** Cleaning corrects or removes records that are broken or invalid. A record that fails cleaning cannot safely enter the pipeline. Apply cleaning strategies only when needed and justify every decision.

**Business objectives:** BO-1, BO-2, BO-3

| Issue | Cleaning Strategy | Justification | Log Requirement |
|---|---|---|---|
| Duplicate invoice_id | Remove duplicates; keep first occurrence | Duplicate rows inflate revenue (BO-1) | Count removed |
| Negative quantity | Drop rows with quantity < 1 | Cannot interpret negative units sold | Count dropped |
| Negative price | | | |
| NULL customer_id | | | |
| Null-like invoice_date | | | |
| Future invoice_date | | | |
| Inaccurate total_revenue | | | |

After cleaning, print a before/after summary:

- Row count before and after
- Null counts per column before and after
- Number of rows dropped per reason

> *Save the cleaned dataset as data/processed/retail_clean.csv before proceeding to Transformation.*

### e. Transformation

**Definition:** Transformation restructures valid data to prepare it for the analytical model. Unlike Cleaning, this step runs unconditionally on all records that survived cleaning — it never drops rows.

**Business objectives:** BO-2, BO-3

Apply the following transformations:

- **Standardize country:** map all variants to title case using a lookup dict: {'colombia': 'Colombia', 'CO': 'Colombia', 'ecuador': 'Ecuador', …}
- **Parse invoice_date:** convert all surviving date strings to pandas datetime (handle YYYY/MM/DD and DD-MM-YYYY variants); extract year, month, day_of_week as separate columns
- **Cast customer_id:** convert float64 to Int64 (nullable integer) after sentinel fill
- **Normalize product:** strip whitespace and apply title case to ensure uniform casing
- **Feature engineering:** create revenue_bin column categorizing total_revenue into Low / Medium / High using quantile-based bins

> *Save the transformed dataset as data/processed/retail_transformed.csv. This file feeds both the post-transformation validation (Task f) and the dimensional model (Task g).*

### f. Post-Transformation Validation (GE)

**Business objectives:** BO-1, BO-2, BO-3, BO-4

The purpose of this suite is different from Task b. Here, the Expectation Suite is a quality contract — all expectations must PASS. If any fail, the pipeline must not proceed to loading.
Reuse the same GE suite from Task b and run it against the transformed dataset. Additionally, add expectations that validate the new transformation-specific columns:

- invoice_date parsed as datetime type with no nulls
- month column in range [1, 12]
- country values only in {Colombia, Ecuador, Peru, Chile}
- revenue_bin only in {Low, Medium, High}
- total_revenue > 0 for all rows
- invoice_id unique with no nulls

Build a comparison table:

| Expectation | Raw (pass %) | Clean (pass %) | Status | Dimension |
|---|---|---|---|---|
| expect_column_values_to_be_unique(invoice_id) | ~77% | 100% | RESOLVED | Uniqueness |
| expect_column_values_to_be_between(quantity, min=1) | ~97% | 100% | RESOLVED | Validity |
| expect_column_values_to_be_between(price, min=0.01) | | | | |
| expect_column_values_to_be_in_set(country, …) | | | | |
| Custom: total_revenue == qty × price | | | | |
| expect_column_values_to_match_regex(date, YYYY-MM-DD) | | | | |
| … (your additional expectations) | … | … | … | … |

Compute and report:

- Data Quality Score (input) = passing expectations / total × 100
- Data Quality Score (output) = passing expectations / total × 100

*Deliverables: Updated Expectation Suite JSON; Data Docs report for clean data; comparison table and DQ Scores in PDF.*

### g. Dimensional Modeling

**Business objectives:** BO-1, BO-2, BO-3
Design a star schema for the retail analytics scenario. The schema must support all KPIs in Task i.
Use the transformed dataset as the source for all tables.
Required Tables

| Table | Type | Key Columns | Source Column(s) |
|---|---|---|---|
| dim_product | Dimension | product_id (surrogate PK), product_name | product |

| | | | |
|---|---|---|---|
| dim_customer | Dimension | customer_id (PK), customer_key | customer_id |
| dim_location | Dimension | location_id (surrogate PK), country | country |
| dim_date | Dimension | date_id (PK: YYYYMMDD int), full_date, year, month, month_name, day_of_week | invoice_date |
| fact_sales | Fact | sale_id (surrogate PK), invoice_id, product_id (FK), customer_id (FK), location_id (FK), date_id (FK), quantity, price, total_revenue | all columns |

Granularity

One row in fact_sales = one transaction line (one invoice, one product, one customer, one date).

Design Requirements

- All foreign keys in fact_sales must reference valid primary keys in their dimension tables.
- dim_date must be fully generated for all dates in 2023 (not just dates present in the dataset).
- Surrogate keys must be integers, not the original source values.
- Include a model description document (reports/model_description.md) with an ERD diagram or table layout.

*Deliverable: Star schema design diagram or description in PDF report; Python script that generates all dimension and fact DataFrames from the transformed dataset.*

### h. Load

Load all tables into a local SQLite database (data_warehouse.db) following the correct load order to maintain referential integrity:

1. Load dim_date first (no foreign keys)
2. Load dim_product, dim_customer, dim_location (no foreign keys)
3. Load fact_sales last (all FKs must resolve)

### i. Business Analysis

Answer the question: "How are sales evolving over time, across products, customers, and regions?"

**Note:** KPIs and visualizations may be produced by querying the SQLite Data Warehouse or directly from the transformed DataFrame. Clearly state which approach you used.

Required KPIs and visualizations (one per Business Objective):

| BO | KPI | Visualization | Notes |
|---|---|---|---|
| BO-1 | Total revenue (sum of total_revenue) | Bar chart: total revenue per country | Exclude duplicate / negative rows — show impact of cleaning |
| BO-1 | Average transaction value | Box plot: total_revenue distribution by product | Highlight outliers |

| | | | |
|---|---|---|---|
| BO-2 | Monthly revenue trend | Line chart: monthly revenue Jan–Dec 2023 | Use parsed month column from dim_date |
| BO-2 | Peak day of week by transaction volume | Bar chart: transaction count by day of week | Use day_of_week from dim_date |
| BO-3 | Top 3 products by total revenue | Horizontal bar chart: revenue by product | Sorted descending |
| BO-3 | Sales distribution by country | Pie or choropleth: revenue share by country | Use standardized country names |
| BO-4 | Data Quality Score before vs. after cleaning | Side-by-side bar: DQ score input vs. output | Values from Task f comparison table |

All visualizations must be saved as PNG files in the reports/ folder.

> *Deliverable: visualization scripts in src/; PNG files in reports/; KPI values and chart screenshots in PDF report.*

## 8. Project Structure (suggested)

```
project/
├── data/
│   ├── raw/                  # retail_etl_dataset_enriched.csv (original, never modified)
│   └── processed/            # retail_clean.csv, retail_transformed.csv, data_warehouse.db
├── src/
│   ├── extract.py            # Task a: load + profiling
│   ├── validate_input.py     # Task b: GE input suite
│   ├── quality_analysis.py   # Task c: DQ issues table + policy proposal
│   ├── clean.py              # Task d: cleaning
│   ├── transform.py          # Task e: transformation
│   ├── validate_output.py    # Task f: GE output suite + comparison
│   ├── dimensional_model.py  # Task g: build dimension + fact tables
│   ├── load_dw.py            # Task h: load to SQLite DW
│   ├── analysis.py           # Task i: KPIs + visualizations
│   └── main.py               # Orchestrator: runs a → i in sequence
├── great_expectations/       # GE context, suites, validations
├── reports/
│   ├── quality_report.md     # Written DQ findings
│   ├── model_description.md  # Star schema ERD and table descriptions
│   └── *.png                 # Visualization outputs
├── requirements.txt
└── README.md                 # Setup and run instructions
```

## 9. Final Deliverables

**a. PDF Report**

A structured report containing all of the following sections in order:

1. Profiling summary (Task a)
2. Input validation results — failure rates per expectation (Task b)
3. Data Quality Issues table (Task c.1)
4. Data Quality Policy Proposal (Task c.2)
5. Cleaning Actions with justification and before/after summary (Task d)
6. Transformation steps applied and rationale (Task e)
7. Output validation results — comparison table + DQ Scores before and after (Task f)
8. Star schema design — ERD or table layout with column descriptions (Task g)
9. Referential integrity check results (Task h)
10. Visualizations and KPIs with interpretation (Task i)

**b. Python Project (Git Repository)**

11. All source scripts in src/ (extract.py through analysis.py + main.py)
12. Great Expectations configuration and Expectation Suite JSON files
13. Data Warehouse SQLite file (data/processed/data_warehouse.db)
14. Visualization PNG files (reports/*.png)
15. README.md with step-by-step instructions to reproduce the full pipeline
16. requirements.txt listing all dependencies with pinned versions